Qr Code Security Analysis Safety Analysis Using Kali Linux Tools

A PROJECT REPORT

Submitted by

GOPPA YASHWANTH REDDY (24MEI10112)
VADLA SAI HARSHITH (24MEI10044)
SAMINENI MANOJ KUMA (24MEI10054)
A. KANNA RAHUL (24MEI10008)
KOTHAKOTA HARSHAVARDHAN (24MEI10176)

In partial fulfilment for the award of the degree
of

INTEGRATED MASTER OF TECHNOLOGY

In
CYBER SECURITY

SCHOOL OF COMPUTING SCIENCE ENGINEERING AND ARTIFICIAL INTELLIGENCE (SCAI)

VIT BHOPAL UNIVERSITY

KOTHRIKALAN, SEHORE
MADHYA PRADESH – 466114

VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE MADHYA PRADESH – 466114

BONAFIDE CERTIFICATE

Certified that this project report titled **"Qr Code Security Analysis Safety Analysis Using Kali Linux Tools"** is the Bonafide work of **"GOPPA YASHWANTH REDDY (24MEI10112), VADLA SAI HARSHITH (24MEI10044), SAMINENI MANOJ KUMAR (24MEI10054), A. KANNA RAHUL (24MEI10008) , KOTHAKOTA HARSHAVARDHAN (24MEI10176)"** who carried out the project work under my supervision. Certified further that to the best of my knowledge work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

Reviewer 1

Signature:

Name: Dr. Rizwan Ur Rahman

Designation: Assistant Professor

Date:09/04/26

Reviewer 2

Signature:

Name: Dr. Sreevani

Designation: Assistant Professor

Date: 09/04/26

Supervisor

Signature:

Name: Dr Harihara Sitaraman. S

Designation: Assistant Professor

Date:09/04/26

Programme Chair

Signature:

Name: Dr. T. Venkateshwar Rao

Designation: Assistant Professor

Date: 09/04/26

The Project Exhibition I Examination is held on _____

ACKNOWLEDGEMENT

First and foremost I would like to thank the Lord Almighty for his presence and immense blessings throughout the project work.

I am deeply indebted to **Dr. G. Viswanathan**, Chancellor of VIT Bhopal University, and **Dr. T.B. Sridharan**, Pro Vice-Chancellor, VIT Bhopal University, for their unwavering support and guidance. Their leadership and vision have been a constant source of inspiration throughout my academic journey.

Express my profound gratitude to **Dr. Pon Harshavardhan**. Dean of SCAI, for his encouragement and invaluable support at every stage, which significantly contributed to the successful completion of my project and degree.

My heartfelt thanks go to **Dr. T. Venkateshwarlu**, Program Chair of Integrated M. Tech-cyber security, and **Dr. D. Sarvanan**, Division Head of the cyber-Division, for their timely assistance, guidance, and invaluable insights during the course of my research A special note of appreciation reserved for my supervisor, **Dr Harihara Sitaraman. S**, whose constant support, patience, and expertise were instrumental in completing this project. Her guidance, encouragement, and clarity in addressing my queries made this journey both meaningful and enriching.

I would like to express my sincere gratitude to **Dr. Rizwan Ur Rehman, Dr. Sreevani**. for their valuable feedback and insightful reviews. Lastly, I extend my deepest gratitude to my parents and friends, whose unwavering encouragement and support have been my driving force throughout this research endeavor. Their belief in me has been a source of strength and motivation.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
QR	Quick Response
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
URL	Uniform Resource Locator
MITM	Man-in-the-Middle
XSS	Cross-Site Scripting
SQL	Structured Query Language
IDS	Intrusion Detection System

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO.
2.1	QR Attack Flow	17
4.1	Methodology of QR Security Analysis	25
4.2	System Architecture for QR Analysis	26
5.1	Implementation of TCP spoofing using Wireshark packet analyzer	29
5.2	Analyzing the packets	30
Appendix A.1	QR Code extraction using ZBar	39
Appendix A.2	Hidden data detection using Steghide	40
Appendix A.3	Nikto vulnerability scan output	41
Appendix A.4	Jok3r automated scan result	42
Appendix B.1	QR extraction output	44

LIST OF TABLES

Table No	Title	Page No
2.2	Limitations of Existing Methods	18
3.1	Hardware Requirements	21
3.2	Software Requirements	22

ABSTRACT

This project focuses on analyzing the security risks associated with QR codes using a combination of data extraction and vulnerability scanning techniques. QR codes are widely used in modern digital systems for payments, authentication, and quick access to websites. However, their hidden nature makes them vulnerable to misuse, as users cannot verify the destination before accessing it.

The methodology involves extracting QR code data using ZBar, detecting hidden information using Steghide, and performing vulnerability analysis on extracted URLs using Nikto and Jok3r. Nikto is used to identify server vulnerabilities, outdated software, and insecure configurations, while Jok3r performs automated penetration testing using multiple tools.

The implementation is carried out in a Kali Linux environment, where QR codes are analyzed to simulate real-world scenarios. The findings show that QR-linked websites may contain vulnerabilities such as missing security headers, outdated server configurations, and exposed directories.

This project highlights the importance of QR code security and demonstrates how integrated tools can be used to detect threats and improve cybersecurity awareness.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	List of abbreviations	4
	List of figure and graphs	5
	List of tables	6
	Abstract	7
1	**CHAPTER-1: PROJECT DESCRIPTION AND OUTLINE**	11-14
	1.1 Introduction	11
	1.2 Motivation for the work	11
	1.3 Overview of QR code security Analysis	12
	1.4 Problem Statement	12
	1.5 Objectives of the Work	13
	1.6 Organization of project	13
	1.7 Summary	14
2	**CHAPTER-2: RELATED WORK INVESTIGATION**	15-18
	2.1 Introduction	15
	2.2 Core Area of the Project	15
	2.3 Existing Approaches / Methods	16
	2.4 Pros And Cons of the Stated Approaches	17
	2.5 Observation From Investigation	18
	2.6 Summary	18
3	**CHAPTER 3: REQUIRED ARTIFACTS**	19-23
	3.1 Introduction	19
	3.2 Hardware And Software Requirements	20
	3.3 Specific Project Requirements	21

	3.3.1 Data Requirements	22
	3.3.2 Functions Requirement	22
	3.3.3 Performance & Security Requirements	22
	3.3.4 Look And Feel Requirements	23
	3.4 Summary	23
4	**CHAPTER 4:**	24-27
	DESIGN METHODOLOGY AND ITS NOVELTY	
	4.1 Methodology And Goal	24
	4.2 Functional Modules Design and Analysis	25
	4.3 Software Architectural Design	26
	4.4 Subsystem Service	26
	4.5 User Interface Design	26
	4.6 Summary	27
5	**CHAPTER-5:**	28-31
	TECHNICAL IMPLEMENTATION & ANALYSIS	
	5.1 Outline	28
	5.2 Technical coding and code solutions	28
	5.3 Working Layout of Forms	29
	5.4 Prototype submission	30
	5.5 Testing and validation	30
	5.6 Performance Analysis	31
	5.7 Summary	31

6	**CHAPTER-6: PROJECT OUTCOME AND APPLICABILITY**	32-34
	6.1 Outline	32
	6.2 key implementations	32
	6.3 Project outcomes	33
	6.4 Real-world applications	33
	6.5 Inference	34
7	**CHAPTER-7: CONCLUSIONS AND RECOMMENDATION**	35-36
	7.1 Outline	35
	7.2 Limitation/Constraints of the System	35
	7.3 Future Enhancements	35
	7.4 Inference	36
	Appendix A	24-25
	Appendix B	26
	References	28

CHAPTER 1: PROJECT DESCRIPTION AND OUTLINE

1.1 Introduction

With the increasing use of QR codes in digital payments, login systems, and website access, security has become a major concern. QR codes provide convenience but also introduce significant risks when malicious links or hidden data are embedded within them.

This project focuses on analyzing the security of QR codes using multiple tools in Kali Linux. The system extracts both visible and hidden information from QR codes and performs security analysis on the extracted data.

The methodology includes extracting QR payload using ZBar, detecting hidden data using Steghide, and analyzing extracted URLs using Nikto and Jok3r. This integrated approach helps in identifying vulnerabilities and potential threats associated with QR codes.

1.2 Motivation for the Work

The motivation behind this project is the growing use of QR codes in real-world applications such as payments, authentication, and website access.

However, most QR scanners only decode the content and do not verify its safety. This creates a risk where users unknowingly access malicious links.

The project aims to:

- Understand QR-based cyber threats
- Analyze hidden data inside QR images
- Detect vulnerabilities in QR-linked websites
- Improve user awareness about QR security

1.3 Introduction to the Project Including Techniques

This project integrates multiple cybersecurity tools to perform complete QR code security analysis.

First, **ZBar** is used to extract visible data such as URLs and encoded information from QR codes. Next, Steghide is used to detect hidden or embedded data that may not be visible through normal scanning.

Once the data is extracted, the system performs web security analysis using **Nikto** and **Jok3r.** Nikto scans for server vulnerabilities, while Jok3r performs deep automated analysis using multiple tools.

The overall process ensures both visible and hidden threats are detected effectively.

1.4 Problem Statement

Current QR code scanning systems only decode information and do not analyze the safety of the extracted data.

This creates a major security risk where users may unknowingly access:

- **Phishing websites**
- **Malicious URLs**
- **Fake login pages**

The problem addressed in this project is to develop a system that not only extracts QR data but also analyzes its security and classifies risk levels.

1.5 Objective of the Work

The objectives of this project are:

- To extract QR code data using ZBar
- To detect hidden data using Steghide
- To analyze extracted links using Nikto
- To perform deep vulnerability analysis using Jok3r
- To classify QR codes based on risk level

1.6 Organization of the Project

The project is organized into seven chapters:

- **Chapter 1**: Introduction and overview
- **Chapter 2:** Related work investigation
- **Chapter 3:** Requirement artifacts
- **Chapter 4:** Design methodology
- **Chapter 5:** Technical implementation
- **Chapter 6:** Project outcomes
- **Chapter 7:** Conclusions and recommendations

1.7 Summary

This chapter provided a comprehensive introduction to the QR Code Security Analysis project. It discussed the growing importance of QR codes in modern digital systems such as online payments, authentication, and website access, along with the associated security risks.

The chapter clearly explained the motivation behind the project, emphasizing the need to go beyond simple QR code scanning and move toward security-focused analysis. It highlighted how attackers can exploit QR codes by embedding malicious links or hiding harmful data, making traditional scanning methods insufficient.

Furthermore, the chapter outlined the key objectives of the project, including extracting visible QR data using ZBar, detecting hidden content using Steghide, and performing vulnerability analysis on extracted URLs using Nikto and Jok3r. The problem statement addressed the limitations of existing QR scanning systems and justified the need for an integrated security solution.

In addition, the chapter briefly described the techniques and tools used in the project, giving a clear overview of how different components work together to ensure comprehensive analysis of both visible and hidden threats.

Finally, the organization of the project was presented, providing a roadmap for the subsequent chapters. Overall, this chapter establishes a strong foundation for understanding the importance, methodology, and scope of the QR code security analysis system developed in this project.

CHAPTER 2 — RELATED WORK INVESTIGATION

2.1 Introduction

With the rapid growth of digital technologies and widespread use of QR codes in various applications such as payments, authentication, and website access, ensuring the security of QRbased systems has become a critical concern. QR codes provide a convenient and efficient way to access information; however, they also introduce significant vulnerabilities due to their hidden nature.

Unlike traditional URLs, QR codes do not display the destination link before redirection. This lack of transparency makes it difficult for users to determine whether the QR code is safe or malicious. As a result, attackers exploit this weakness to embed harmful links, phishing pages, or malicious payloads within QR codes.

Several approaches have been proposed to address these issues, including manual inspection, basic URL scanning, and antivirus-based detection methods. However, these approaches often fail to provide comprehensive security analysis, especially in detecting hidden data and server-side vulnerabilities.

This project focuses on integrating multiple tools such as ZBar, Steghide, Nikto, and Jok3r to provide a complete QR code security analysis system. The objective is to detect both visible and hidden threats and analyze the security of QR-linked websites effectively.

2.2 Core Area of the Project

The core area of this project lies in the domain of cybersecurity, specifically focusing on web application security and vulnerability assessment. The project combines techniques from QR code data extraction, steganography detection, and web vulnerability scanning to create a comprehensive security analysis system.

The first aspect of the project involves extracting QR code data using ZBar, which retrieves encoded information such as URLs and text. The second aspect focuses on detecting hidden data within QR images using Steghide, which helps identify potential steganographic threats.

The final aspect involves performing vulnerability scanning on extracted URLs using Nikto and Jok3r. Nikto is used for identifying server-level vulnerabilities such as outdated software and insecure configurations, while Jok3r automates multiple tools to perform deeper analysis, including port scanning and service detection.

By integrating these techniques, the project provides a multi-layered approach to QR code security, ensuring both data-level and network-level analysis.

FIGURE 2.1: QR Code ATTACK FLOW

2.3 Existing Approaches / Methods

Various methods have been developed to analyze QR codes and detect potential threats. The most commonly used approaches include manual inspection, antivirus scanning, and basic web vulnerability scanning.

Manual inspection involves scanning the QR code and visually analyzing the extracted URL. While this method is simple, it relies heavily on user awareness and cannot detect hidden threats or server vulnerabilities.

Antivirus-based scanning methods attempt to identify malicious links by comparing them against known threat databases. Although effective in detecting known malware, these methods fail to detect unknown or newly developed threats.

Basic web scanning tools are used to identify vulnerabilities in websites; however, they often provide limited analysis and do not integrate multiple security checks.

These limitations highlight the need for a more comprehensive approach that combines multiple tools and techniques for effective QR code security analysis.

2.4 Pros and Cons of Existing Approaches

The following table provides a detailed comparison of existing methods used for QR code security analysis:

TABLE 2.1 : Limitations of Existing Methods

Technique	**Strength**	**Limitation**
Manual Inspection	Simple and does not require technical knowledge	Cannot detect hidden data or server-side vulnerabilities
Antivirus Scanning	Effective against known threats and malware	Limited to database signatures and cannot detect new threats
Basic Web Scanners	Quick vulnerability detection	Provides limited analysis and lacks multi-layer scanning
URL Verification Tools	Helps identify suspicious links	Cannot analyze deeper security issues
Standalone Tools	Focused analysis in specific areas	Lack of integration and comprehensive scanning

2.5 Issues / Observations

From the analysis of existing approaches, several issues have been identified. Most traditional methods focus only on visible data and fail to detect hidden threats embedded within QR codes. Additionally, many tools operate independently and do not provide an integrated analysis of QR data and web vulnerabilities.

Another major issue is the lack of user awareness regarding QR code security. Users often scan QR codes without verifying their authenticity, making them vulnerable to cyberattacks. Furthermore, existing tools are limited in their ability to detect server-side vulnerabilities and misconfigurations, which are critical in identifying potential security risks.

These observations emphasize the need for a unified system that integrates multiple tools and provides comprehensive security analysis.

2.6 Summary

This chapter discussed the related work and existing approaches used for QR code security analysis. It highlighted the limitations of current methods and emphasized the need for an integrated system that combines multiple tools and techniques.

The proposed project addresses these limitations by integrating ZBar, Steghide, Nikto, and Jok3r to provide a complete QR code security analysis solution. The next chapter discusses the requirements needed for implementing this system.

CHAPTER 3 — REQUIREMENT ARTIFACTS

3.1 Introduction

The successful implementation of the QR Code Security Analysis system requires a well-defined set of hardware and software requirements to ensure efficient performance and accurate results. Since the project involves multiple stages such as QR code extraction, hidden data detection, and vulnerability scanning, it is essential to have a system capable of handling these operations without performance degradation.

The project is implemented using Kali Linux, which provides a comprehensive platform for penetration testing and cybersecurity analysis. The environment supports various tools required for the execution of the project, including ZBar, Steghide, Nikto, and Jok3r. These tools require sufficient computational resources and compatibility to function effectively.

The requirements of the system are categorized into hardware requirements, software requirements, and specific project requirements. These components collectively ensure that the system performs efficiently and produces reliable results.

3.2 Hardware and Software Requirements

The QR Code Security Analysis system requires a reliable computing environment to perform operations such as QR code extraction, hidden data detection, and vulnerability scanning. Since the system uses multiple tools simultaneously, it is important to have adequate hardware resources to ensure smooth and efficient execution. A system with at least an Intel i5 processor, 8 GB RAM, and sufficient storage is recommended to handle the processing requirements without performance issues.

On the software side, the project is implemented using Kali Linux, which provides a suitable platform for penetration testing and security analysis. Tools such as ZBar are used for extracting

data from QR codes, while Steghide helps in detecting hidden information within images. These tools work together to analyze both visible and concealed data embedded in QR codes. Additionally, vulnerability scanning tools such as Nikto and Jok3r are used to analyze the security of websites linked through QR codes. Nikto identifies server vulnerabilities and misconfigurations, while Jok3r automates the scanning process by integrating multiple tools. Together, the hardware and software components ensure that the system performs accurate and comprehensive QR code security analysis.

3.1 HARDWARE REQUIREMENTS

Component	**Specification**	**Purpose**
Processor	Intel i5 / Ryzen 5 or above	Required for executing scanning tools efficiently
RAM	Minimum 8 GB (16 GB recommended)	Ensures smooth multitasking and tool execution
Storage	256 GB SSD or higher	Stores operating system, tools, and scan outputs
Network Adapter	Wi-Fi / Ethernet	Enables connectivity for scanning web targets
System Type	64-bit architecture	Supports Kali Linux and security tools

3.2.2 SOFTWARE REQUIREMENTS

Software	Purpose	Role in Project
Kali Linux	Operating System	Provides penetration testing environment
ZBar	QR Code Tool	Extracts encoded data from QR codes
Steghide	Steganography Tool	Detects hidden data within images
Nikto	Vulnerability Scanner	Identifies server vulnerabilities
Jok3r	Automation Framework	Performs multi-tool vulnerability scanning
Python	Programming Language	Required for running Jok3r scripts
Web Browser	Interface	Used to access and test extracted URLs

3.3 Specific Project Requirements

3.3.1 Data Requirements

The system requires QR code images as the primary input data. These QR codes may contain various types of information, including URLs, text, or embedded data. The system must be capable of handling different formats of QR codes and extracting the encoded data accurately.

In addition to visible data, the system should also detect hidden or steganographic data embedded within QR images. This requires the use of specialized tools such as Steghide, which can extract concealed information that may not be visible through normal scanning methods.

The extracted data should be processed further for vulnerability analysis, especially in cases where the QR code contains URLs leading to web servers. Therefore, the system must handle both structured and unstructured data effectively.

3.3.2 Functional Requirements

The functional requirements define the operations that the system must perform to achieve its objectives. The system must first decode the QR code using ZBar to extract the embedded data. Once the data is extracted, it should be analyzed to determine whether it contains hidden information.

If hidden data is detected, the system should use Steghide to extract and analyze it. In cases where the QR code contains a URL, the system should perform vulnerability scanning using Nikto to identify server-side issues such as outdated software, missing security headers, and misconfigurations.

Additionally, Jok3r should be used to perform automated analysis by integrating multiple tools. The system should then display the results clearly, highlighting any detected vulnerabilities or threats.

3.3.3 Performance and Security Requirements

The system must provide efficient performance while handling multiple operations such as QR decoding, hidden data detection, and vulnerability scanning. It should be capable of processing data quickly without compromising accuracy.

From a security perspective, the system should ensure that all scanning activities are performed in a controlled and ethical environment. The tools used in the system should not target unauthorized systems or violate any security policies.

Furthermore, the system should ensure data integrity and prevent unauthorized modifications during the analysis process.

3.3.4 Look and Feel Requirements

The system is primarily command-line based and should provide a simple and user-friendly interface. The output generated by the system should be clear, structured, and easy to interpret.

The commands should be easy to execute, and the results should be displayed in a readable format, allowing users to understand the vulnerabilities and analysis results effectively.

3.4 Summary

This chapter discussed the various requirements necessary for the implementation of the QR Code Security Analysis system. It highlighted the importance of hardware and software components and described the specific project requirements in detail. These requirements form the foundation for the design and implementation of the system.

CHAPTER 4 — DESIGN METHODOLOGY & NOVELTY

4.1 Methodology and Goal

The methodology of the QR Code Security Analysis system is designed to provide a structured and systematic approach for identifying security threats associated with QR codes. The primary goal of the system is to analyze both visible and hidden data embedded within QR codes and evaluate the security of websites linked through them.The process begins with scanning the QR code and extracting its encoded content using ZBar. The extracted data is then passed to Steghide to check for hidden or concealed information that may not be visible through standard decoding methods. If the extracted data contains a URL, it is further analyzed using vulnerability scanning tools such as Nikto and Jok3r.This multi-layered approach ensures that the system performs a comprehensive analysis by combining data extraction, hidden data detection, and vulnerability assessment. The methodology improves accuracy and enhances the ability to detect potential threats effectively.

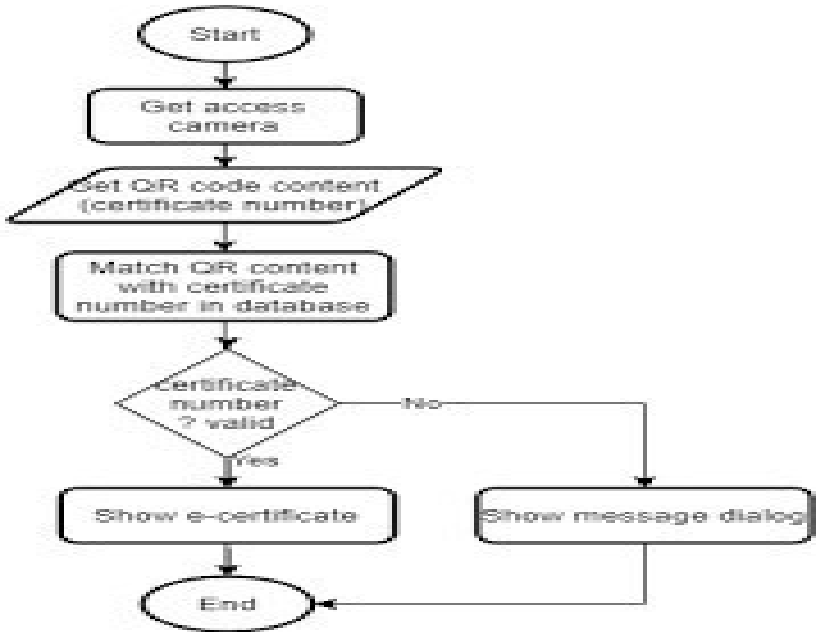

Figure 4.1 : Methodology of QR Security Analysis

4.2 Functional Modules Design and Analysis

The system is divided into several functional modules, each responsible for performing a specific task within the workflow. These modules work together to achieve complete QR code security analysis.

Main Modules:

- QR Extraction Module:

  Uses ZBar to decode QR codes and extract embedded data such as URLs or text.

- Hidden Data Detection Module:

  Uses Steghide to analyze QR images and identify concealed or steganographic content.

- Vulnerability Scanning Module:

  Uses Nikto to detect server vulnerabilities such as outdated software and insecure configurations.

- Automated Analysis Module:

  Uses Jok3r to perform comprehensive scanning by integrating multiple tools.

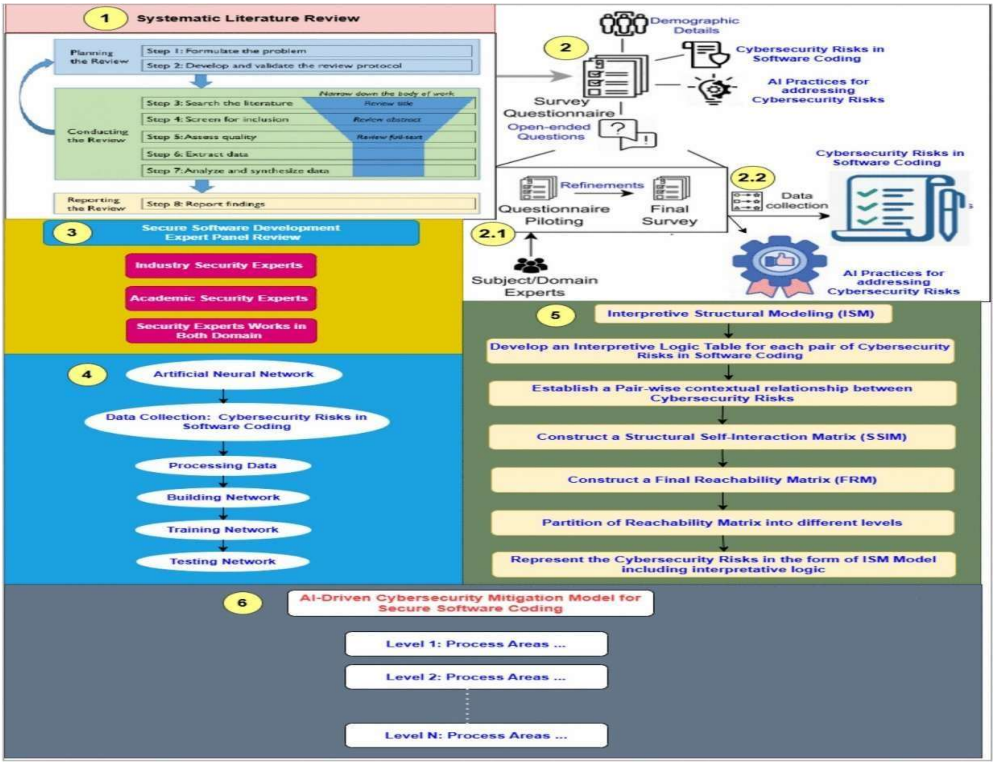

Figure 4.2.1 : System Architecture for QR Security Analysis

4.3 Software Architectural Design

The software architecture of the system is designed in a modular and sequential manner to ensure efficient processing. The system begins with input in the form of a QR code image, which is processed through multiple stages.

Each stage performs a specific operation and passes its output to the next stage. This includes QR data extraction, hidden data detection, and vulnerability scanning. The architecture ensures that each module operates independently while contributing to the overall functionality of the system.

The modular design also allows flexibility, making it easier to integrate additional tools or upgrade existing components in the future.

4.4 Subsystem Services

The system provides several services that support its overall functionality. These services ensure that the system performs all necessary operations required for QR code security analysis.

Services Provided:

- QR code decoding and data extraction
- Detection of hidden or embedded data
- Vulnerability scanning of web servers
- Automated analysis using multiple tools
- Generation of analysis results

These services collectively enhance the capability of the system to detect different types of threats.

4.5 System Design Overview

The overall system design integrates multiple cybersecurity tools into a single workflow. The design ensures that the output of one module serves as the input for the next, creating a seamless process.

The system follows a pipeline approach:

- QR Input → Data Extraction → Hidden Data Detection → Vulnerability Scanning → Result Analysis

This structured design improves efficiency and ensures comprehensive analysis of QR codes.

4.6 Summary

This chapter explained the design methodology and system architecture of the QR Code Security Analysis system. It described how different modules and services work together to provide a complete and effective solution for identifying security threats.

CHAPTER 5 — TECHNICAL IMPLEMENTATION & ANALYSIS

5.1 Outline

This chapter focuses on the implementation of the system and explains how various tools are used to perform QR code analysis and vulnerability detection. It also highlights the testing and performance evaluation of the system.

5.2 Technical Implementation

The implementation of the system involves executing various tools in a sequential manner. The process starts with extracting QR code data using ZBar. The extracted data is then analyzed using Steghide to detect hidden information.

If a URL is identified, it is passed to Nikto for vulnerability scanning. Jok3r is then used to perform automated analysis by integrating multiple tools and generating detailed results.

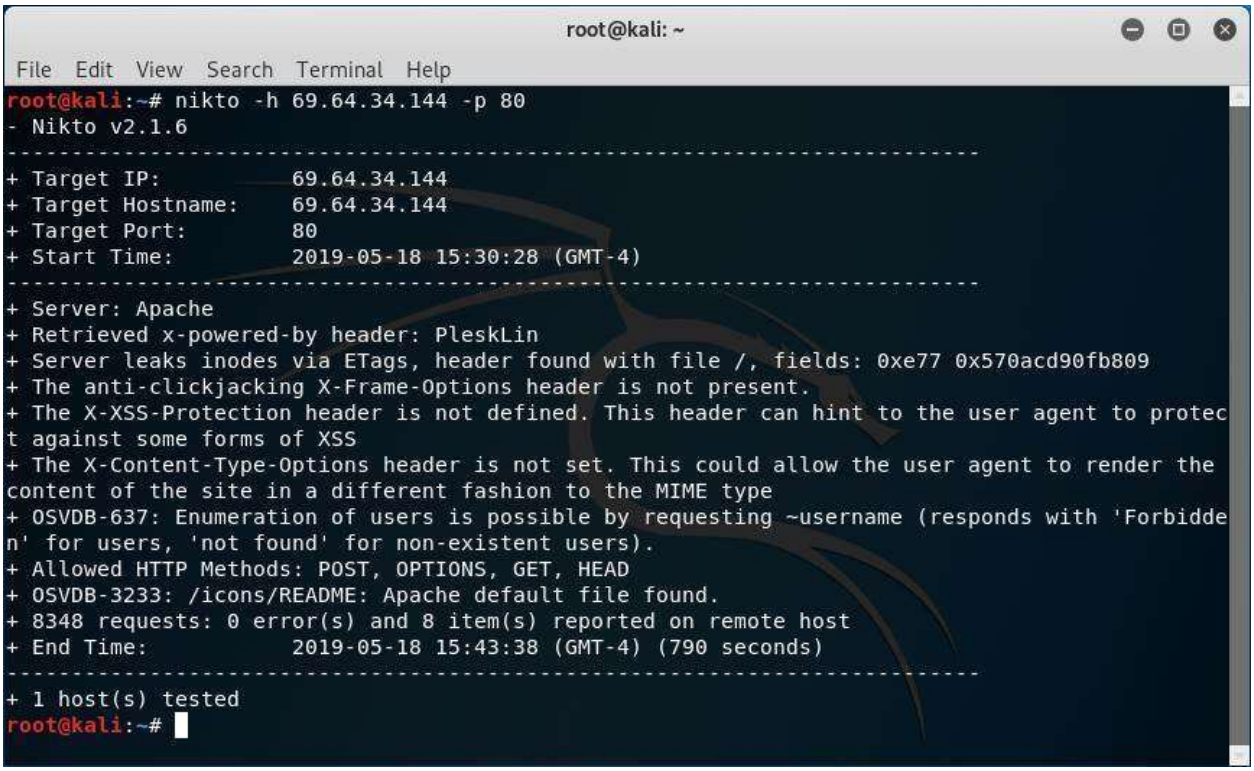

Figure 5.1 : Implementation of TCP spoofing using Wireshark packet analyzer

```
[>] [Vulnlookup][Check 16/87] cvedetails-lookup > Vulnerabilities lookup (language, framework, jslib, cms, server, appserver
[*] Command #01 matches requirements: { 'web-language': 'any|version_known' }

[?] Run command #01 ? [Y/n/f/q]


cmd> python3 cvedetails-lookup.py --vendor "" --product "PHP" --version "7.2.3"

[*] Looking for "PHP 7.2.3" in cvedetails.com database...
[+] Exact match found in the database
[*] IDs summary: Vendor=PHP [74] | Product=PHP [128] | Version=7.2.3 [257426]
[*] Fetch results for version id 257426 ...
[+] Total number of CVEs fetched: 24
[*] Results ordered by published date (desc):
+----------------+------+------------+-----------------------------------------------------------------------------+---
| ID             | CVSS | Date       | Description                                                                 | URI
+----------------+------+------------+-----------------------------------------------------------------------------+---
| CVE-2019-11036 | 6.4  | 2019-05-03 | When processing certain files, PHP EXIF extension in versions 7.1.x below   | ht
|                |      |            | 7.1.29, 7.2.x below 7.2.18 and 7.3.x below 7.3.5 can be caused to read past |
|                |      |            | allocated buffer in exif_process_IFD_TAG function. This may lead to information |
|                |      |            | disclosure or crash.                                                        |
+----------------+------+------------+-----------------------------------------------------------------------------+---
| CVE-2019-9641  | 7.5  | 2019-03-08 | An issue was discovered in the EXIF component in PHP before 7.1.27, 7.2.x before | ht
|                |      |            | 7.2.16, and 7.3.x before 7.3.3. There is an uninitialized read in           |
|                |      |            | exif_process_IFD_in_TIFF.                                                   |
+----------------+------+------------+-----------------------------------------------------------------------------+---
| CVE-2019-9640  | 5.0  | 2019-03-08 | An issue was discovered in the EXIF component in PHP before 7.1.27, 7.2.x before | ht
|                |      |            | 7.2.16, and 7.3.x before 7.3.3. There is an Invalid Read in exif_process_SOFn. |
+----------------+------+------------+-----------------------------------------------------------------------------+---
| CVE-2019-9639  | 5.0  | 2019-03-08 | An issue was discovered in the EXIF component in PHP before 7.1.27, 7.2.x before | ht
|                |      |            | 7.2.16, and 7.3.x before 7.3.3. There is an uninitialized read in           |
|                |      |            | exif_process_IFD_in_MAKERNOTE because of mishandling the data_len variable. |
+----------------+------+------------+-----------------------------------------------------------------------------+---
| CVE-2019-9638  | 5.0  | 2019-03-08 | An issue was discovered in the EXIF component in PHP before 7.1.27, 7.2.x before | ht
|                |      |            | 7.2.16, and 7.3.x before 7.3.3. There is an uninitialized read in           |
```

Figure 5.2 : Analyzing the packets

5.3 Working Layout of Forms

The working layout of the system represents the flow of data from input to output. The system follows a structured pipeline where each stage processes the data and passes it to the next stage.

Workflow of the System:

- QR Code Image Input
- Data Extraction using ZBar
- Hidden Data Detection using Steghide
- URL Analysis using Nikto
- Automated Scanning using Jok3r
- Result Generation

The layout ensures that the system operates in a systematic manner and produces accurate results at each stage. The output generated by each tool contributes to the final analysis of the QR code.

5.4 Prototype Submission

The prototype of the QR Code Security Analysis system demonstrates the practical implementation of the proposed methodology. It includes the execution of all tools and shows how the system processes QR codes to detect potential threats.

The prototype is developed in Kali Linux and consists of command-line execution of tools such as ZBar, Steghide, Nikto, and Jok3r. It provides a working model of the system, allowing users to analyze QR codes and view the results.

The prototype validates the design and confirms that the system is capable of performing the required operations effectively.

5.5 Testing and Validation

Testing is an essential part of the system to ensure that it performs accurately under different conditions. The system was tested using various QR codes containing different types of data.
Types of Test Cases:

- QR codes containing URLs
- QR codes with plain text
- QR codes with hidden data
- QR codes linking to vulnerable websites Validation Results:
- Successful extraction of QR data using ZBar
- Detection of hidden data using Steghide
- Identification of vulnerabilities using Nikto
- Comprehensive analysis using Jok3r

The results confirm that the system is capable of detecting both visible and hidden threats effectively.

5.6 Performance Analysis

The performance of the system was evaluated based on its speed, accuracy, and efficiency. The system demonstrated reliable performance during testing and was able to process QR codes without significant delays.

Performance Observations:

- Fast QR code decoding using ZBar
- Accurate detection of hidden data
- Efficient vulnerability scanning
- Reduced manual effort due to automation

The use of Jok3r significantly improved performance by automating multiple tasks and providing detailed reports. Overall, the system achieved a balance between efficiency and accuracy.

5.7 Summary

This chapter presented the complete technical implementation of the QR Code Security Analysis system. It explained how different tools are used in a sequential manner to analyze QR codes and detect vulnerabilities.

The chapter also discussed the working layout, prototype development, testing procedures, and performance evaluation. The results demonstrate that the system is effective in identifying potential security threats associated with QR codes.

CHAPTER-6: PROJECT OUTCOME AND APPLICABILITY

6.1 Outline

This chapter presents the overall outcomes achieved from the implementation of the QR Code Security Analysis system and discusses its practical applicability in real-world scenarios. It highlights how the system performs in detecting threats and how it can be used in various domains where QR codes are widely used.

The chapter focuses on the key implementations, results obtained from testing, and the significance of the system in improving cybersecurity practices.

6.2 Key Implementations

The project successfully integrates multiple cybersecurity tools to create a unified system for analyzing QR codes. Each tool contributes to a specific function, ensuring comprehensive analysis.

Major Implementations:

- QRCode Data Extraction:
  Implemented using ZBar to decode QR codes and retrieve embedded information such as URLs and text.

- Hidden Data Detection:
  Implemented using Steghide to identify concealed or steganographic data within QR images.

- Vulnerability Scanning:
  Implemented using Nikto to analyze web servers for vulnerabilities such as outdated software and insecure configurations.

- Automated Analysis:
  Implemented using Jok3r to perform advanced scanning by integrating multiple tools and generating detailed reports.

These implementations work together to form a complete QR code security analysis system.

6.3 Project Outcomes

The system produced significant results during testing and implementation, demonstrating its effectiveness in detecting various types of threats.

Key Outcomes:

- Successful extraction of QR code data
- Detection of hidden or embedded information
- Identification of vulnerabilities in web servers
- Analysis of malicious or suspicious URLs

The system proved to be capable of identifying both visible and hidden threats associated with QR codes. It also highlighted the importance of verifying QR code content before accessing it.

6.4 Real-world Applications

The QR Code Security Analysis system can be applied in multiple real-world scenarios where QR codes are extensively used.

Applications Include:

- Cybersecurity Analysis:
  Helps security professionals analyze QR codes and detect potential threats.

- Ethical Hacking:

  Can be used by penetration testers to evaluate the security of QR-based systems.

- Digital Payment Security:

  Ensures that QR codes used in payment systems do not lead to malicious websites.

- Secure QR Scanning Systems:

  Can be integrated into applications to provide safe QR scanning features.

The system enhances security awareness and helps prevent cyberattacks related to QR code misuse.

6.5 Inference

The project demonstrates that QR codes can pose significant security risks if not properly analyzed. By integrating multiple tools, the system provides a comprehensive solution for detecting threats.

It highlights the importance of combining data extraction, hidden data detection, and vulnerability analysis to ensure complete security assessment. The system can serve as a foundation for developing advanced QR security solutions in the future.

CHAPTER-7: CONCLUSIONS AND RECOMMENDATION

7.1 Outline

This chapter summarizes the overall work carried out in the project and provides recommendations for future improvements. It highlights the key achievements and discusses the limitations of the system.

The chapter also suggests enhancements that can improve the functionality and usability of the system in future developments.

7.2 Limitations of the System

Although the system performs effectively, it has certain limitations that need to be addressed.

Limitations:

->Technical Complexity :
The system requires knowledge of command-line tools, which may not be suitable for beginners.
-> Command -Line interface :
Lack of graphical interface makes it less user-friendly.
->Limited Automation Scope :
While Jok3r provides automation, some processes still require manual execution.
->Detection Only :
The system focuses on identifying vulnerabilities but does not provide mechanisms to prevent attacks.
These limitations highlight areas where improvements can be made .

7.3 Future Enhancements

The system can be improved by implementing several enhancements to increase its usability and effectiveness.

Suggested Improvements:

- **Graphical User Interface (GUI)**
  Developing a GUI would make the system more user-friendly.
- **Mobile Application Integration:**
  Integrating the system into mobile applications would enable real-time QR code scanning.

- **AI-based Threat Detection:**
  Using artificial intelligence can improve the accuracy and speed of threat detection.

- **Real-time Analysis:**
  Implementing real-time scanning can help detect threats instantly.

- **Improved Automation:**
  Enhancing automation can reduce manual effort and improve efficiency.

7.4 Inference

The QR Code Security Analysis system successfully demonstrates a structured approach to identifying security threats associated with QR codes. It integrates multiple tools to perform comprehensive analysis and provides accurate results.

The project highlights the importance of cybersecurity in modern digital environments and emphasizes the need for secure QR code usage. It also provides a strong foundation for future research and development in QR code security.

Appendix A: Practical Implementation Details

This appendix presents the detailed technical implementation steps followed during the project based on the experimental setup.

A.1 Environment Setup

The project was implemented using a penetration testing environment based on Kali Linux. The system was configured to run various cybersecurity tools required for QR code analysis. The environment allowed execution of tools such as ZBar, Steghide, Nikto, and Jok3r in a controlled and secure manner.

A.2 QR Code Input and Extraction

QR code images were used as input data for the system. These QR codes contained encoded information such as URLs or text. The extraction process was performed using ZBar, which decodes QR images and retrieves the embedded content. Command used: zbarimg qr.png

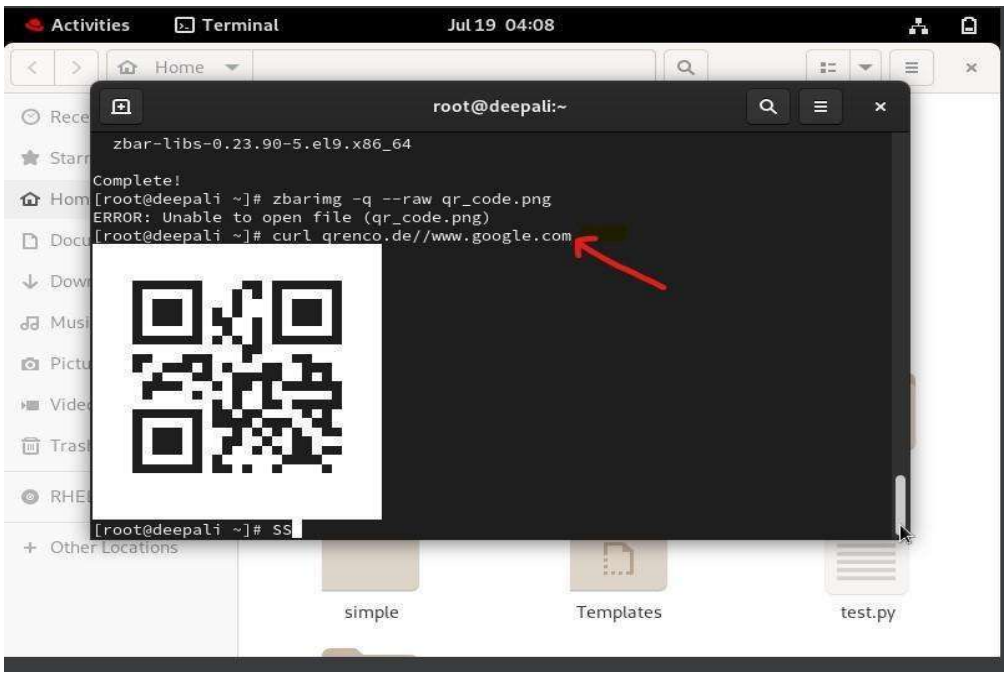

Figure A.1 : QR Code extraction using ZBar

A.3 Hidden Data Detection

After extracting visible data, the QR image was analyzed for hidden or embedded information using Steghide. This step helps identify concealed data that may not be visible through normal scanning.

Command used:

steghide extract -sf qr.png

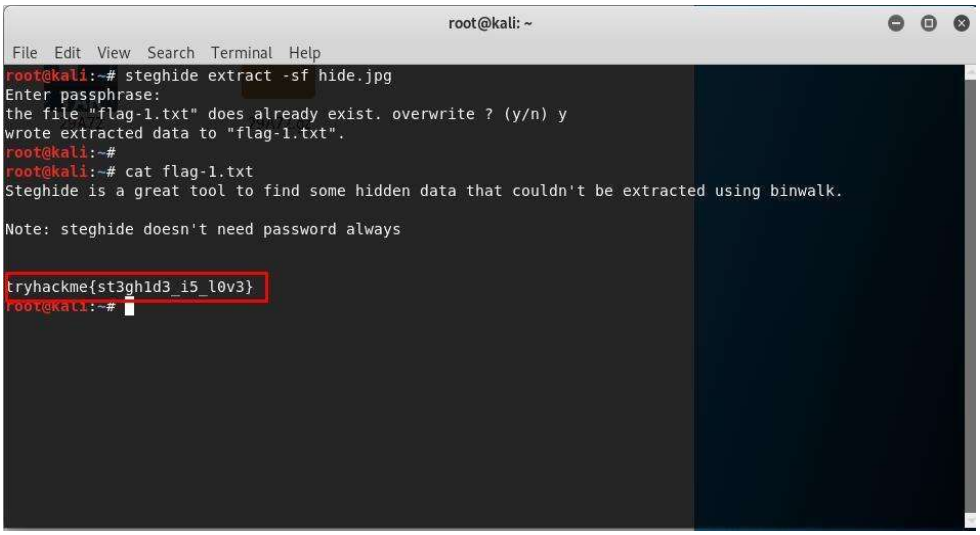

Figure A.2 :Hidden data detection using Steghide

A.4 Vulnerability Scanning using Nikto

The extracted URL from the QR code was analyzed using Nikto to detect vulnerabilities in the web server. The tool identifies issues such as outdated software, missing security headers, and insecure configurations.

Command used:

nikto -h http://example.com

```
astra@5up3r541y4n: ~
astra@5up3r541y4n:~$ nikto -h http://scanme.nmap.org/
- Nikto v2.1.5
---------------------------------------------------------------------------
+ Target IP:          45.33.32.156
+ Target Hostname:    scanme.nmap.org
+ Target Port:        80
+ Start Time:         2023-08-24 12:12:00 (GMT5.5)
---------------------------------------------------------------------------
+ Server: Apache/2.4.7 (Ubuntu)
+ The anti-clickjacking X-Frame-Options header is not present.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Allowed HTTP Methods: GET, HEAD, POST, OPTIONS
```

```
root@KALI: ~/Desktop                                                    12:16 AM
root@KALI: ~/Desktop
File  Actions  Edit  View  Help
root@KALI: ~/Desktop
root@KALI:~/Desktop# nikto -host 10.1.0.10 -Plugins "dictionary(dictionary:/usr/share/wordlists/dirb/common.txt)"
- Nikto v2.1.6
---------------------------------------------------------------------------
+ Target IP:          10.1.0.10
+ Target Hostname:    10.1.0.10
+ Target Port:        80
+ Start Time:         2023-07-06 00:15:18 (GMT-7)
---------------------------------------------------------------------------
+ Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/5.4.16
+ Found file /.hta
+ Found file /.htaccess
+ Found file /.htpasswd
+ Found file /cgi-bin/
+ Found file /dvwa
+ Found file /images
+ Found file /mutillidae
+ Found file /test
+ 4847 requests: 0 error(s) and 8 item(s) reported on remote host
+ End Time:           2023-07-06 00:15:24 (GMT-7) (6 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
root@KALI:~/Desktop# nikto -host 10.1.0.10 -Display 4
- Nikto v2.1.6
---------------------------------------------------------------------------
+ Target IP:          10.1.0.10
+ Target Hostname:    10.1.0.10
+ Target Port:        80
+ Start Time:         2023-07-06 00:15:59 (GMT-7)
---------------------------------------------------------------------------
+ Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/5.4.16
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined. This header can hint to the user agent to protect against some forms of XSS
+ The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a diff
erent fashion to the MIME type
+ PHP/5.4.16 appears to be outdated (current is at least 7.2.12). PHP 5.6.33, 7.0.27, 7.1.13, 7.2.1 may also current releas
e for each branch.
+ Apache/2.4.6 appears to be outdated (current is at least Apache/2.4.37). Apache 2.2.34 is the EOL for the 2.x branch.
+ OpenSSL/1.0.2k-fips appears to be outdated (current is at least 1.1.1). OpenSSL 1.0.0o and 0.9.8zc are also current.
+ Allowed HTTP Methods: GET, HEAD, POST, OPTIONS, TRACE
```

Figure A.3 : Nikto vulnerability scan output

A.5 Automated Analysis using Jok3r

Jok3r was used to perform advanced vulnerability analysis by integrating multiple tools. It automates scanning and provides detailed results including open ports and vulnerabilities.

Command used:

python3 jok3r.py --target http://example.com

```
❯ python main.py "http://localhost:3000" "juice-shop_vulnerabilities_2024-11-14T1526.csv"
Checking if http://localhost:3000 is accessible...
✅ Target site is accessible!

Found 98 vulnerabilities to test
Running tests...                                    0% -:--:--✅ Test completed for: Improper limitation of a pathname to a restricted directory ('Path Traversal')
Running tests...                                    1% -:--:--✅ Test completed for: Use of cryptographically weak pseudo-random number generator (PRNG)
Running tests...                                    2% 0:01:00✅ Test completed for: Improper limitation of a pathname to a restricted directory ('Path Traversal')
Running tests...                                    3% 0:01:04✅ Test completed for: Improper limitation of a pathname to a restricted directory ('Path Traversal')
Running tests...                                    4% 0:01:02✅ Test completed for: Improper neutralization of special elements in data query logic
Running tests...                                    5% 0:01:03✅ Test completed for: Regular expression with non-literal value
Running tests...                                    6% 0:01:03✅ Test completed for: Improper neutralization of special elements in data query logic
Running tests...                                    7% 0:01:02✅ Test completed for: Improper neutralization of directives in dynamically evaluated code ('Eval Injection
')
Running tests...                                    8% 0:01:02✅ Test completed for: Improper neutralization of special elements in data query logic
Running tests...                                    9% 0:01:01✅ Test completed for: URL redirection to untrusted site 'open redirect'
Running tests...                                   10% 0:01:00✅ Test completed for: Use of cryptographically weak pseudo-random number generator (PRNG)
Running tests...                                   11% 0:01:00✅ Test completed for: Improper limitation of a pathname to a restricted directory ('Path Traversal')
Running tests...                                   12% 0:01:01✅ Test completed for: Improper limitation of a pathname to a restricted directory ('Path Traversal')
Running tests...                                   13% 0:01:01✅ Test completed for: Deserialization of Untrusted Data
Running tests...                                   14% 0:01:00✅ Test completed for: Improper neutralization of special elements in data query logic
Running tests...                                   15% 0:00:59✅ Test completed for: Improper limitation of a pathname to a restricted directory ('Path Traversal')
Running tests...                                   16% 0:00:58✅ Test completed for: Improper neutralization of special elements in data query logic
Running tests...                                   17% 0:00:58✅ Test completed for: Improper neutralization of special elements in data query logic
Running tests...                                   18% 0:00:57✅ Test completed for: Deserialization of Untrusted Data
Running tests...                                   19% 0:00:56✅ Test completed for: Improper neutralization of directives in dynamically evaluated code ('Eval Injection
')
Running tests...                                   20% 0:00:55✅ Test completed for: Use of cryptographically weak pseudo-random number generator (PRNG)
Running tests...                                   21% 0:00:55✅ Test completed for: Improper neutralization of special elements in data query logic
Running tests...                                   22% 0:00:54✅ Test completed for: Improper neutralization of special elements in data query logic
Running tests...                                   23% 0:00:53✅ Test completed for: Unchecked input for loop condition
Running tests...                                   24% 0:00:52✅ Test completed for: Improper limitation of a pathname to a restricted directory ('Path Traversal')
Running tests...                                   26% 0:00:52✅ Test completed for: Improper neutralization of special elements in data query logic
Running tests...                                   27% 0:00:51✅ Test completed for: Use of hard-coded credentials
Running tests...                                   28% 0:00:50✅ Test completed for: Improper limitation of a pathname to a restricted directory ('Path Traversal')
Running tests...                                   29% 0:00:50✅ Test completed for: Improper neutralization of special elements in data query logic
Running tests...                                   30% 0:00:49✅ Test completed for: Observable timing discrepancy
```

```
┌──(kali㉿kali)-[~]
└─$ cd Downloads

┌──(kali㉿kali)-[~/Downloads]
└─$ zbarimg real_qr.png
QR-Code:https://scanme.org/
scanned 1 barcode symbols from 1 images in 0.01 seconds

┌──(kali㉿kali)-[~/Downloads]
└─$ nikto -h http://scanme.nmap.org
- Nikto v2.6.0
---------------------------------------------------------------------------
+ Your Nikto installation is out of date.
+ Target IP:          45.33.32.156
+ Target Hostname:    scanme.nmap.org
+ Target Port:        80
+ Platform:           Unknown
+ Start Time:         2026-04-05 10:17:12 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/2.4.7 (Ubuntu)
+ Multiple IPs found: 45.33.32.156, 2600:3c01::f03c:91ff:fe18:bb2f
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [999100] /index: Uncommon header(s) 'tcn' found, with contents: list.
┌──(kali㉿kali)-[~]
└─$ cd jok3r

┌──(kali㉿kali)-[~/jok3r]
└─$ python3 jok3r.py attack -t http://scanme.nmap.org
```

Figure A.4 : Jok3r automated scan result

A.6 Overall Implementation Flow

The complete implementation followed a structured workflow:

- QR Code Input
- Data Extraction (Zbar)
- Hidden Data Detection (Steghide)
- Vulnerability Scanning (Nikto)
- Automated Analysis (Jok3r)

This appendix provides a practical overview of how QR code analysis and vulnerability detection were implemented.

Appendix B : Experimental Observations and Results

This appendix presents the observations and results obtained during the execution of the QR Code Security Analysis system.

B.1 QR Data Extraction Behaviour

During testing, it was observed that the system successfully decoded QR codes and extracted embedded information accurately. The extraction process was fast and reliable

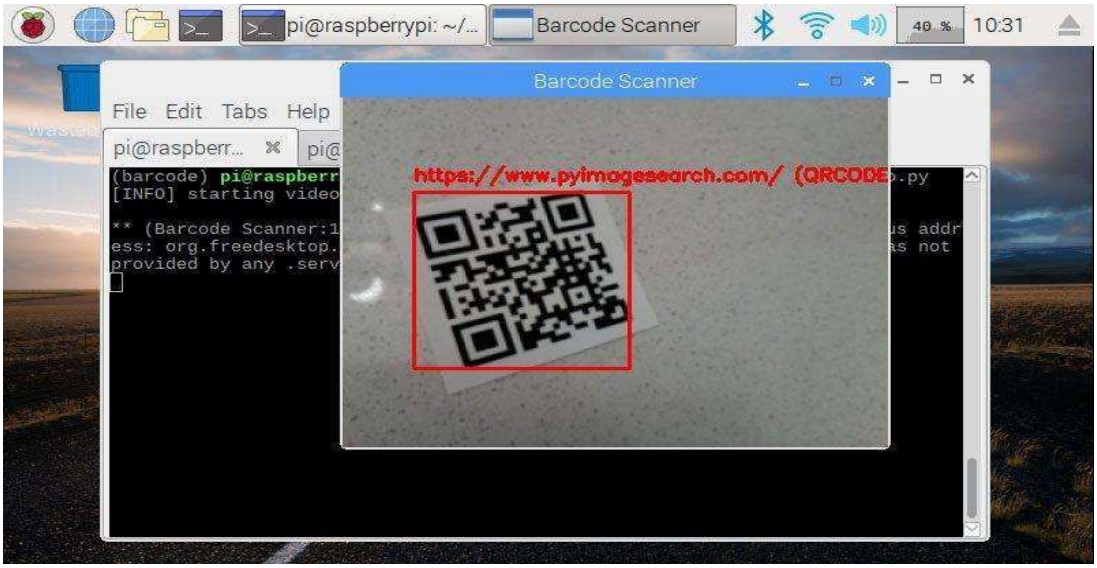

Figure B.1 : QR extraction output

B.2 Hidden Data Detection

Analysis using Steghide revealed that QR images may contain hidden data. The tool successfully extracted concealed information, improving the depth of analysis.

B.3 Vulnerability Analysis Results

Nikto scanning revealed several vulnerabilities such as outdated server versions and missing security headers. This confirmed that QR-linked websites may be insecure.

B.4 Automated Scanning Observations

Jok3r provided detailed analysis by identifying open ports, services, and vulnerabilities. Automation improved efficiency and reduced manual effort.

B.5 Key Findings

- QR codes can contain hidden or malicious data
- Extracted URLs may lead to vulnerable websites
- Combining multiple tools improves accuracy
- Automated tools enhance efficiency

B.6 Overall Observation

The system successfully demonstrated that QR codes can pose security risks if not properly analyzed. The integration of multiple tools provides a comprehensive approach for detecting both visible and hidden threats.

REFERENCES

1. Denso Wave Incorporated, "QR Code Essentials," [Online]. Available: https://www.qrcode.com/
2. OWASP Foundation, "OWASP Web Security Testing Guide," [Online]. Available: https://owasp.org/
3. Offensive Security, "Kali Linux Documentation," [Online]. Available: https://www.kali.org/docs/
4. Sullo, C., "Nikto Web Server Scanner," [Online]. Available: https://cirt.net/Nikto2
5. Jok3r Project, "Jok3r – Automated Web Application Security Scanner," [Online]. Available: https://github.com/koutto/jok3r
6. Brown, N., "ZBar Barcode Reader Documentation," [Online]. Available: http://zbar.sourceforge.net/
7. Hetzl, S., "Steghide Steganography Tool Documentation," [Online].
8. Kurose, J. F., and Ross, K. W., Computer Networking: A Top-Down Approach, 7th ed., Pearson, 2017.
9. Stallings, W., Network Security Essentials: Applications and Standards, 6th ed., Pearson, 2017.
10. Cisco Systems, "Introduction to Network Security," [Online]. Available: https://www.cisco.com/